

Museo de Baler — Technical Reference & Defense Sheet

As of 5 October 2026

Read this first: claims to drop before the defense

The original brief named several technologies the system does not use, so every answer in this sheet was checked against the code on 2026-10-05. If you say "Angular" or "Sanctum" and a panelist opens the repository, the defense is over. Use the right-hand column.

Brief said	The system actually does	Say this instead
Angular / Ionic Framework	Vanilla JavaScript, no build step. Static files in <code>public/visitor/</code> (<code>index.html</code> , <code>js/app.js</code> , <code>sw.js</code> , <code>manifest.json</code>)	"A framework-free PWA, so the phone downloads no framework runtime."
Laravel Sanctum	A custom <code>visitor</code> bearer-token guard registered in Laravel's auth system. Tokens are stored as SHA-256 hashes	"A stateless bearer-token guard on Laravel's own auth layer."
Cloudflare Tunnels for hosting	<code>cloudflared</code> was used only to test camera and GPS on a phone from the laptop. Production runs on Apache 2.4 (Ubuntu, Hostinger VPS) with a Let's Encrypt certificate. Cloudflare is not in front of it	"Tunnel for development; a hardened VPS with TLS for production."
Nginx	Apache 2.4	Apache
"Why no ML / image recognition?"	It has image recognition: an on-device Teachable Machine (TensorFlow.js) classifier, about 3 MB, plus a server-side perceptual-hash (pHash) fallback. QR is the primary path	Defend QR as primary and ML as an optional assist (Q1)
Separate domains for PWA and admin	Same domain, separate paths: <code>/visitor/</code> and the panel. Separated by auth guard, middleware and CSP, not by domain	Q3 answer below
Geofence checked on every exhibit scan	Geofence (150 m default) is checked at attendance check-in. Exhibit unlock requires today's desk clearance, not per-scan GPS	Q7 answer below
Group code <code>BALER-XXXX</code>	6 characters from a 31-letter alphabet with no 0/O or 1/I, no prefix, valid only on the group's visit date, capped at the headcount	"A 6-character same-day join code."
Children ≤ 7 , seniors, PWD hard-coded	Free and discounted categories are rows the Administrator configures (<code>admission_discounts</code>)	"Categories are configurable; the museum sets who enters free."
Visitor QR pass with Approved → Checked-In	There is no visitor ticket QR. The app says "It is a guide, not a ticket." Admission is bound to the signed-in account	Q10 answer below
<code>transactions</code> + <code>visitor_sessions</code> tables	<code>admission_payments</code> (append-only ledger), <code>visits</code> , <code>visit_groups</code> , <code>visitors</code> , <code>attendances</code> , <code>scans</code>	Q11 answer below
IndexedDB	Not used. <code>localStorage</code> , <code>sessionStorage</code> , and Cache Storage (service worker)	Drop IndexedDB
Web Audio API	Audio guides play through the HTML <code><audio></code> element	"HTML5 audio"
Background Sync	No Background Sync API. A <code>localStorage</code> scan queue (max 200) is flushed when the network returns	Q6 answer below
Argon2	bcrypt, 12 rounds	bcrypt only
"Dynamic" exhibit QR tokens	Exhibit QR codes are static deep links (ECC level H). Access control is on the server, not in the code	Q7 answer below

Brief said	The system actually does	Say this instead
"< 50 ms scan verification"	Not measured	Don't quote a number you haven't measured
(missing) Google Gemini API	Used for drafting exhibit translations and narration. Optional: the museum runs without it, and no visitor data is sent	Add it to your stack slide

Your own [docs/LIMITATIONS.md](#) still says there is no visitor password reset and no email verification. Both were built on 2026-10-03, so update that file before panelists read it.

1. Core technology stack

One Laravel 12 application on PHP 8.3 and MySQL has three faces: a server-rendered staff panel (Blade), a JSON API under [/api/v1](#), and an installable visitor PWA served as static files from [/visitor/](#) that talks only to that API.

Frontend: visitor PWA ([/visitor/](#))

Component	What is used	Purpose
UI	Vanilla JavaScript, HTML, CSS. No framework, no build step	Small download, nothing to compile on deploy
Camera / QR scanning	getUserMedia via the html5-qrcode library	Live scanning of exhibit QR codes
On-device recognition	TensorFlow.js + Teachable Machine image model (~3 MB, in visitor/model/)	Optional point-the-camera exhibit identification
Location	HTML5 Geolocation API	Attendance check-in inside the museum geofence
Audio guides	HTML5 <audio> element, MP3 files, byte-range served	Narrated exhibit guides
Offline	Service worker sw.js , versioned per release	Shell: stale-while-revalidate. Media: cache-first. API: network-first with last-good fallback
Installability	manifest.json + icons	Add to Home Screen, full-screen launch
Client storage	localStorage (session token, app state, offline scan queue), sessionStorage (per-tab exhibit cache), Cache Storage	Survive reloads and dead zones

Backend and API

Component	What is used	Purpose
Framework	Laravel 12 on PHP 8.3	Routing, validation, ORM, queues, scheduler
API style	RESTful JSON under /api/v1 , versioned	Three tiers: public, signed-in (visitor.auth), signed-in + cleared by the desk (visitor.cleared)
Visitor auth	Custom visitor bearer-token guard; tokens stored SHA-256 hashed	Stateless auth for the phone app
Staff auth	Laravel web session guard, encrypted cookie, HttpOnly , SameSite=Strict	Panel sign-in
Authorization	EnsureRole (Administrator, TourismHead), EnsureVisitorCleared , DesktopOnly , RequirePasswordChange	Role and admission gates

Component	What is used	Purpose
Hardening	<code>SecurityHeaders</code> (CSP, HSTS, frame-deny), <code>SanitizeInput</code> , named rate limiters, <code>AuditLog</code>	Defence in depth
Media URLs	Temporary signed URLs (<code>signed:relative</code>)	Lets <code><audio>/</code> load gated media without a header
Reports	mPDF, PhpSpreadsheet, PhpWord, native CSV	PDF, XLSX, DOCX, CSV exports
QR generation	<code>chillerlan/php-qrcode</code> , SVG, error-correction level H	Printable exhibit labels that tolerate 30% damage

Database

Component	What is used	Purpose
RDBMS	MySQL, session timezone pinned to Asia/Manila	Transactions, foreign keys, unique constraints
ORM	Eloquent, with the query builder for reports	Models such as <code>Visitor</code> , <code>Visit</code> , <code>VisitGroup</code> , <code>AdmissionPayment</code> , <code>Attendance</code> , <code>Scan</code> , <code>Exhibit</code>
Schema control	Laravel migrations, versioned in git	Reproducible schema on every server
Testing	PHPUnit feature tests, plus a Node test for the service worker	Regression safety

2. External APIs, identity and cryptography

The system depends on three outside services (Google OAuth, Gmail SMTP and Google Gemini), and the museum keeps running if any of them is down. Passwords, codes and tokens are never stored in plain text.

Service / mechanism	How it works here	Safeguards
Google OAuth 2.0 (Laravel Socialite)	"Continue with Google" goes to <code>/auth/google/callback</code> . The server issues a one-time code, and the PWA exchanges it at <code>POST /api/v1/visitors/google</code> for a bearer token	Token never travels in a URL. <code>google_id</code> is unique. 20 requests/min per IP. The button stays hidden until the OAuth keys are configured
Email OTP: sign-up verification	A 6-digit code is emailed over SMTP. No session token exists until it is typed back	Stored as a hash. Expires in 15 min. 5 wrong tries kill the code. Resend after 60 s. Throttled per account and per IP
Email OTP: password reset	Three steps: code by email, then code for a reset token, then token for a new password	Code and token both hashed. 15 min expiry, 5 attempts, per-account and per-IP throttles
Google Gemini API	Drafts exhibit translations and narration for staff to review	Optional. Sends exhibit text only, never visitor records
Password hashing	bcrypt, cost 12, through Laravel's <code>hashed</code> cast on <code>Staff</code> and <code>Visitor</code>	Visitor policy: length + common-password blocklist (NIST SP 800-63B). Staff must change the issued password at first sign-in
API tokens	Random bearer tokens, stored as SHA-256 hashes	A stolen database dump does not yield usable tokens
Session data	Laravel encrypted sessions (<code>SESSION_ENCRYPT=true</code>)	120 min, ends on browser close, no "remember me"
Backups	XChaCha20-Poly1305 (libsodium), authenticated	A tampered or stolen dump cannot be read

If asked about Argon2: Laravel supports it with a single config change. bcrypt at cost 12 is the framework default and still meets OWASP guidance.

3. Infrastructure, networking and hosting

Production runs at museodebaler.com on an Ubuntu VPS (Hostinger) behind Apache 2.4 with a Let's Encrypt certificate. A Cloudflare Tunnel was used only during development.

Layer	Production (live)	Development / testing
Entry point	DNS points straight to the VPS. The bare domain redirects to <code>/visitor/</code>	<code>cloudflared</code> tunnel to the laptop's Laragon server
TLS	Let's Encrypt, renew by 2026-12-23. HTTP 301s to HTTPS. HSTS for 1 year with <code>includeSubDomains</code>	Tunnel provides a trusted HTTPS certificate
Why HTTPS matters	Service workers, camera (<code>getUserMedia</code>) and geolocation only work in a secure context	Over plain <code>http://192.168.x.x</code> the camera and GPS refuse to start, which is why the tunnel was needed
Web server	Apache 2.4 with per-folder <code>.htaccess</code> (separate CSP for <code>/visitor/</code>)	Apache (Laragon)
CORS	Locked to the app's own origin	Same
Rate limiting	Per-IP and per-account Laravel limiters (e.g. staff login 5 failures/min/IP)	Same
Deployment	<code>deploy.sh</code> : build a new release folder from GitHub <code>main</code> , migrate, cache config, then switch the <code>current</code> symlink (blue/green, instant rollback)	<code>git push</code>
Backups	Nightly encrypted MySQL dump plus a media archive	Manual

Media storage

Asset	Format and handling
Exhibit photos	The original is archived and never sent to phones. Two WebP derivatives: ~1280 px display (quality 78) and ~400 px thumbnail. WebP is about a third smaller than JPEG. Falls back to JPEG if the server lacks WebP support
Audio guides	MP3, served with HTTP byte ranges so phones can seek. Cached on the phone after first play
Recognition model	<code>model.json</code> , <code>weights.bin</code> , <code>metadata.json</code> (~3 MB), retrained from the staff panel
QR labels	SVG, generated when an exhibit is created
Reports	Generated on demand (PDF, XLSX, DOCX, CSV). A nightly CSV batch is kept for 90 days in <code>storage/app/exports/</code>
Gated media	Served only through temporary signed URLs issued to cleared visitors

4. Internal modules and business logic

These are modules inside one Laravel application (Models, Services, Middleware), not separately deployed microservices. If a panelist says "microservices", call them "modules".

Module	Where it lives	How it works
Transaction reference generator <code>MDB-YYYYMMDD-NNNN</code>	<code>AdmissionPayment::nextReference()</code>	One sequence per day, in Manila time. The read takes a row lock (<code>lockForUpdate</code>) inside a DB transaction, so two desks saving at once queue instead of colliding. A unique index on <code>reference</code> is the backstop, with up to 3 retries. Past 9999 the number simply grows a digit
Consolidated receipt	<code>AdmissionPayment::forVisitor()</code> / group payment	One ledger row per payment. The payer's fee plus a ₱0.00 line for each free category share one transaction number. Line items are kept in a <code>breakdown</code> snapshot

Module	Where it lives	How it works
Append-only ledger	<code>admission_payments</code>	Rows are written once and never edited. A correction is a separate <code>refund</code> row with its own number. The earnings report sums the ledger, so a past day's takings never change
Group join code	<code>VisitGroup::freshJoinCode()</code> , <code>joinable()</code>	The desk registers a group and gets a 6-character code from <code>ABCDEFGHIJKMNPQRSTUVWXYZ23456789</code> (no 0/O, 1/I). Members type it in the app (<code>POST /visitors/me/group</code>). Valid only on the group's visit date and capped at its headcount. Paying once clears every member
Group fee rule	<code>VisitGroup::feeFor()</code>	Locals enter free. Otherwise fee × the paying count the desk enters, so mixed parties pay only for non-locals
Companions / free add-ons	<code>Visitor::setCompanions()</code> , <code>admission_discounts</code>	Dependents without phones are stored as visitor rows linked to the primary (<code>companion_of</code>), so they still count in demographics. Categories (child, senior, PWD, etc.) and their discount are set by the Administrator, not hard-coded. Replaced atomically in one DB transaction
Admission gate	<code>Visitor::clearance()</code> , <code>EnsureVisitorCleared</code>	Exhibits, scans, recognition and the survey need a token whose visitor the desk cleared today (paid, or ID-verified as exempt). Yesterday's clearance does not count
Geofence check-in	<code>GeofenceService</code> , <code>POST /api/v1/attendance</code>	Haversine distance from the museum pin. Default radius 150 m, set from the panel. GPS accuracy widens the margin up to 2× the radius. Outside: refused with the distance shown. Inside: one <code>attendances</code> row per visitor per day (unique key)
Stay telemetry	<code>PATCH /api/v1/attendance/{id}</code>	The app updates <code>exited_at</code> (last seen) and <code>duration_mins</code> . Only the larger duration is kept, so a late or out-of-order update cannot shorten a visit
Exhibit QR + scan log	<code>ExhibitQr</code> , <code>ScanController</code> , <code>scans</code> log	Each exhibit QR is a static deep link to that exhibit. Each scan logs exhibit, visitor, language, method (QR or camera) and server time. Scans made in a dead zone queue on the phone and post when the signal returns
Image recognition	<code>visitor/model/</code> (on-device), <code>ImageSearch</code> (pHash fallback), <code>Recognition</code> (training)	Staff photograph exhibits and press Train in the panel. The browser trains the model and saves it back to the server
Staff attendance	<code>StaffAttendance</code> , <code>StaffSchedule</code> , <code>AttendanceCorrection</code>	Clock-in/out against weekly schedules. Present/absent per day. Corrections need Tourism approval
Reports and audit	<code>ReportBuilder</code> , four exporters, <code>AuditLog</code>	Logbook, DTR, visitors, exhibits, feedback (ARTA CSM), earnings, audit trail. One dataset feeds CSV / XLSX / DOCX / PDF. Every staff action is logged after the role check passes

5. Panel defense Q&A

Each answer leads with the claim, then the mechanism, then the honest limit. Where the original question assumed something false, the answer corrects it politely. Panels reward that more than they punish it.

Q1. Storage, bandwidth, and why QR instead of ML/computer vision

Answer. We store one original per exhibit photo as an archive copy that is never sent to phones. Visitors receive two WebP derivatives: about 1280 px for the detail view and 400 px for lists. WebP is roughly a third

smaller than JPEG at the same quality. The service worker caches media cache-first, and the exhibit list is revalidated by ETag, so an unchanged list costs a 304 with no body.

We did not reject machine learning; we put it in the right place. QR is the primary path because decoding is instant, runs on any phone, needs no network to decode, and has no false matches. Recognition is an optional assist: a ~3 MB Teachable Machine (TensorFlow.js) model runs on the visitor's phone, so the server needs no GPU and handles no images. The server-side fallback is a perceptual hash (pHash), which is CPU-only, with reference hashes cached. Staff retrain the model from the panel.

If pressed: a server-side CNN would put GPU and RAM cost on a museum server for every camera frame from every visitor. Our design moves that cost to the phones that already have it.

Q2. Why PHP/Laravel instead of Python (Django/FastAPI)?

Answer. Laravel gave us, built in, what this project needed: Eloquent ORM, versioned migrations, Socialite for Google sign-in, rate limiters, signed URLs, encrypted sessions, the bcrypt `hashed` cast, a task scheduler for nightly backups and reports, and a feature-test harness. One language serves both the staff panel and the API.

PHP's share-nothing request model means each request starts clean and frees its memory when it ends, so there is no long-running process to leak or restart. PHP on Apache is also the most common stack on Philippine VPS and shared hosting, and the one LGU IT staff are most likely to maintain after turnover.

If pressed: Django or FastAPI would work as well technically. The decision was ecosystem fit, hosting availability and maintainability by the people who inherit it, not raw speed.

Q3. Why separate the visitor PWA from the admin panel?

Correction first: they share one domain and are separated by path, authentication and policy, not by domain.

Answer. The two sides never share credentials. Visitors hold a bearer token that only works on `/api/v1`, and that API is tiered: public, signed-in, and signed-in and cleared by the desk. Staff use a session cookie (`HttpOnly`, `SameSite=Strict`, encrypted) that the API ignores. `DesktopOnly` middleware keeps phones out of the panel entirely, and `EnsureRole` limits each staff role to its own functions (least privilege).

The service worker's scope is `/visitor/`, so it cannot cache a panel page. Gated API answers are `Cache-Control: private` and deleted from the phone on a 401, 403 or sign-out. Each side has its own Content-Security-Policy and its own rate limiters.

If pressed: moving the panel to a subdomain is now a DNS and config change, because the code already treats them as separate applications.

Q4. Why MySQL instead of MongoDB (or PostgreSQL)?

Answer. The core of this system is money and attendance, which need ACID guarantees. Issuing a transaction number takes a row lock inside a database transaction, so two desks cannot issue the same number. Unique constraints enforce business rules at the database level: one attendance per visitor per day, one visit per visitor per day, one row per transaction reference.

Foreign keys link visitors, visits, groups, payments and the staff who recorded them, so a group member cannot point at a group that does not exist. Our reports, such as the logbook, earnings and ARTA CSM, are GROUP BY aggregates, which relational SQL does natively. MongoDB would push those guarantees into application code.

If pressed on PostgreSQL: it would serve equally well. MySQL was chosen for hosting availability and team

familiarity, not because PostgreSQL falls short.

Q5. Why an installable PWA instead of native Android/iOS apps?

Answer. A visit lasts about an hour. Asking a tourist to find, download and install a store app at the entrance loses most of them. With a PWA, the visitor scans the entrance QR and is in the app within seconds, with Add to Home Screen optional.

One codebase serves Android and iOS. Updates ship the moment we deploy: the service worker is versioned per release, so there is no app-store review cycle. Camera, GPS and offline caching are all available to the browser over HTTPS.

Honest limit: iOS clears the cache of sites not used for about a week. For a one-visit app that does not matter, and a returning visitor simply re-downloads about 5 MB.

Q6. What happens offline inside thick museum walls?

Answer. The service worker uses three strategies:

- **App shell:** stale-while-revalidate, so the app opens instantly from cache and updates in the background.
- **Photos and audio:** cache-first, with byte-range support so cached audio can still seek.
- **Exhibit data:** network-first, falling back to the last good copy when the network fails.

Scans made in a dead zone are queued in `localStorage` (up to 200) and posted when the signal returns, so the halls with the worst reception do not look least visited in the analytics.

Honest limit: records that change official numbers (registration, attendance, the ARTA survey) require the network and say so. The survey is never queued, because a duplicate would distort the published CSM percentages. Content a phone never loaded online is not available offline.

Q7. How do you stop someone scanning an exhibit QR from home or social media?

Correction first: the protection is not in the QR code. The code is only a link to an exhibit.

Answer. Exhibit content is served only to a signed-in visitor whom front-desk staff cleared today, after collecting the fee or verifying an exempt ID (`EnsureVisitorCleared`). A photo of the QR on Facebook opens the app to a locked screen. Clearance is checked against today's date, so yesterday's visit unlocks nothing. Editing `localStorage` changes the screen, not the server's answer.

The geofence is used at attendance check-in: the Haversine distance from the museum pin must fall within a configurable radius (150 m default), with a margin for reported GPS accuracy.

Why not GPS on every scan: indoor GPS inside a stone building is often less precise than the 150 m fence, so paying visitors would be refused. It would also break the offline scan queue. GPS is easily spoofed anyway, so a staff member at the desk is the stronger check.

Honest limit: a visitor cleared in the morning keeps access after leaving until midnight. That is accepted because admission is a per-visit fee.

Q8. How do you handle data privacy (RA 10173 / GDPR)?

Answer. We collect only what the museum's mandatory logbook, fee rules and ARTA reporting need: name, age, sex, email, home locality and visit type. Location is read only at check-in, not tracked continuously. No third party receives visitor records; the Gemini AI feature receives exhibit text only.

In transit, everything is HTTPS with HSTS. At rest, passwords use bcrypt (cost 12), API tokens and one-time codes are stored as hashes, sessions are encrypted, and backups use authenticated XChaCha20-Poly1305 encryption. Access is role-based, and every staff action is written to an audit log. A purge command supports a retention policy once the museum adopts one. Companions without phones are recorded by the desk, so they never need an account.

Honest limit: personal data columns are not encrypted individually, because reports must search and group on them. That is mitigated by no public database access, a scoped database user, encrypted backups and host disk encryption, and it is documented as a known limitation.

Q9. Why use Cloudflare Tunnels instead of port forwarding or a public IP?

Correction first: the tunnel was for development. Production runs on a VPS.

Answer. Camera, GPS and service workers only work over HTTPS. A phone testing against a laptop at <http://192.168.x.x> gets none of them. [cloudflared](#) opens an outbound-only connection, so the phone gets a trusted certificate without opening a router port or exposing the laptop's IP.

In production, museodebaler.com runs on Apache with a Let's Encrypt certificate, HTTPS-only redirects, a 1-year HSTS header, per-IP rate limiting and a locked-down CORS policy.

If pressed on DDoS: Cloudflare's proxy is not in front of production today. The app already trusts Cloudflare's IP ranges, so enabling it is a DNS change that would add edge DDoS protection and caching.

Q10. What stops two people from using one screenshotted QR pass?

Correction first: there is no visitor ticket QR to screenshot. The app itself tells visitors "It is a guide, not a ticket."

Answer. Admission is attached to the account, not to an image. Staff clear a specific visitor for a specific date, and the server checks that on every gated request. The database enforces one attendance and one visit per visitor per day with unique constraints, so a repeated check-in returns "already logged" instead of a second entry. Transaction numbers are issued under a row lock with a unique index, so concurrent desks cannot double-issue. Group join codes expire at the end of the visit date and cannot admit more members than the headcount paid for.

Honest limit: two people sharing one login would share one admission. The desk counts heads at entry, which is the same control a paper logbook relies on.

Q11. Doesn't one transaction number per group create redundancy or orphans?

Answer. No. Each entity has its own table: [visitors](#) (people), [visits](#) (one per visitor per day), [visit_groups](#) (the party), and [admission_payments](#) (money). The payment row references visitor, visit, group and recording staff by foreign key, and companions link to their primary visitor through [companion_of](#). Group membership lives on the visit, not copied into the payment, so there is no stacking of duplicate records.

One deliberate denormalization is the ledger's line-item [breakdown](#), which is snapshotted onto the payment row. A financial record must show what was charged at that moment, even if the fee or discount categories change later, just like a printed receipt. The ledger is append-only: corrections are new refund rows, never edits.

Q12. How does the system handle 1,000 visitors on one holiday morning?

Answer. 1,000 visitors over a four-hour morning is about four arrivals a minute, and the front desk is the bottleneck, not the server. After the first load, the app shell, photos, audio and recognition model come from each phone's own cache. The exhibit list is revalidated by ETag, so most refreshes are bodiless 304s.

Recognition runs on the phone, which takes the heaviest work off the server.

Visitor authentication is a stateless bearer token, so more PHP workers, or a second server behind a load balancer, share nothing but the database. Hot query paths are indexed.

Honest limit: we have not run a formal load test. If the panel wants a number, the next step is a k6 or JMeter run against staging. Do not quote a latency figure you have not measured.